4

MACHINE LEARNING 101

Data science is best understood as a partnership between a data scientist and a computer. In chapter 2, we described the process the data scientist follows: the CRISP-DM life cycle. CRISP-DM defines a sequence of decisions the data scientist has to make and the activities he should engage in to inform and implement these decisions. In CRISP-DM, the major tasks for a data scientist are to define the problem, design the data set, prepare the data, decide on the type of data analysis to apply, and evaluate and interpret the results of the data analysis. What the computer brings to this partnership is the ability to process data and search for patterns in the data. Machine learning is the field of study that develops the algorithms that the computers follow in order to identify and extract patterns from data. ML algorithms and techniques are applied primarily during

the modeling stage of CRISP-DM. ML involves a two-step process.

First, an ML algorithm is applied to a data set to identify useful patterns in the data. These patterns can be represented in a number of different ways. We describe some popular representations later in this chapter, but they include decision trees, regression models, and neural networks. These representations of patterns are known as "models," which is why this stage of the CRISP-DM life cycle is known at the "modeling stage." Simply put, ML algorithms create models from data, and each algorithm is designed to create models using a particular representation (neural network or decision tree or other).

Second, once a model has been created, it is used for analysis. In some cases, the structure of the model is what is important. A model structure can reveal what the important attributes are in a domain. For example, in a medical domain we might apply an ML algorithm to a data set of stroke patients and use the structure of the model to identify the factors that have a strong association with stroke. In other cases, a model is used to label or classify new examples. For instance, the primary purpose of a spam-filter model is to label new emails as either spam or not spam rather than to reveal the defining attributes of spam email.

Supervised versus Unsupervised Learning

The majority of ML algorithms can be classified as either *supervised learning* or *unsupervised learning*. The goal of supervised learning is to learn a function that maps from the values of the attributes describing an instance to the value of another attribute, known as the *target attribute*, of that instance. For example, when supervised learning is used to train a spam filter, the algorithm attempts to learn a function that maps from the attributes describing an email to a value (spam/not spam) for the target attribute; the function the algorithm learns is the spam-filter model returned by the algorithm. So in this context the pattern that the algorithm is looking for in the data is the function that maps from the values of the input attributes to the values of the target attribute, and the model that the algorithm returns is a computer program that implements this function. Supervised learning works by searching through lots of different functions to find the function that best maps between the inputs and output. However, for any data set of reasonable complexity there are so many combinations of inputs and possible mappings to outputs that an algorithm cannot try all possible functions. As a consequence, each ML algorithm is designed to look at or prefer certain types of functions during its search. These preferences are known as the algorithm's *learning bias*. The real challenge in using ML is to find the algorithm whose

learning bias is the best match for a particular data set. Generally, this task involves experiments with a number of different algorithms to find out which one works best on that data set.

Supervised learning is "supervised" because each of the instances in the data set lists both the input values and the output (target) value for each instance. So the learning algorithm can guide its search for the best function by checking how each function it tries matches with the data set, and at the same time the data set acts as a supervisor for the learning process by providing feedback. Obviously, for supervised learning to take place, each instance in the data set must be labeled with the value of the target attribute. Often, however, the reason a target attribute is interesting is that it is not easy to directly measure, and therefore it is not possible to easily create a data set of labeled instances. In such scenarios, a great deal of time and effort is required to create a data set with the target values before a model can be trained using supervised learning.

In unsupervised learning, there is no target attribute. As a consequence, unsupervised-learning algorithms can be used without investing the time and effort in labeling the instances of the data set with a target attribute. However, not having a target attribute also means that learning becomes more difficult: instead of the specific problem of searching for a mapping from inputs to output that

The real challenge in using ML is to find the algorithm whose learning bias is the best match for a particular data set.

matches the data, the algorithm has the more general task of looking for regularities in the data. The most common type of unsupervised learning is *cluster analysis*, where the algorithm looks for clusters of instances that are more similar to each other than they are to other instances in the data. These clustering algorithms often begin by guessing a set of clusters and then iteratively updating the clusters (dropping instances from one cluster and adding them to another) so as to increase both the within-cluster similarity and the diversity across clusters.

A challenge for clustering is figuring out how to measure similarity. If all the attributes in a data set are numeric and have similar ranges, then it probably makes sense just to calculate the Euclidean distance (better known as the *straight-line distance*) between the instances (or rows). Rows that are close together in the Euclidean space are then treated as similar. A number of factors, however, can make the calculation of similarity between rows complex. In some data sets, different numeric attributes have different ranges, with the result that a variation in row values in one attribute may not be as significant as a variation of a similar magnitude in another attribute. In these cases, the attributes should be normalized so that they all have the same range. Another complicating factor in calculating similarity is that things can be deemed similar in many different ways. Some attributes are sometimes more important than other attributes, so it might make sense to

weight some attributes in the distance calculations, or it may be that the data set includes nonnumeric data. These more complex scenarios may require the design of bespoke similarity metrics for the clustering algorithm to use.

Unsupervised learning can be illustrated with a concrete example. Imagine we are interested in analyzing the causes of Type 2 diabetes in white American adult males. We would begin by constructing a data set, with each row representing one person and each column representing an attribute that we believe are relevant for the study. For this example, we will include the following attributes: an individual's height in meters and weight in kilos, the number of minutes he exercises per week, his shoe size, and the likelihood that he will develop diabetes expressed as a percentage based on a number of clinical tests and lifestyle surveys. Table 2 illustrates a snippet from this data

Table 2 Diabetes Study Data Set

ID	Height (meters)	Weight (kilograms)	Shoe Size	Exercise (minutes per week)	Diabetes (% likelihood)
1	1.70	70	5	130	0.05
2	1.77	88	9	80	0.11
3	1.85	112	11	0	0.18
...					

set. Obviously, other attributes could be included—for example, a person's age—and some attributes could be removed—for example, shoe size, which wouldn't be particularly relevant in determining whether someone will develop diabetes. As we discussed in chapter 2, the choice of which attributes to include and exclude from a data set is a key task in data science, but for the purposes of this discussion we will work with the data set as is.

An unsupervised clustering algorithm will look for groups of rows that are more similar to each other than they are to the other rows in the data. Each of these groups of similar rows defines a cluster of similar instances. For instance, an algorithm can identify causes of a disease or disease comorbidities (diseases that occur together) by looking for attribute values that are relatively frequent within a cluster. The simple idea of looking for clusters of similar rows is very powerful and has applications across many areas of life. Another application of clustering rows is making product recommendations to customers. If a customer liked a book, song, or movie, then he may enjoy another book, song, or movie from the same cluster.

Learning Prediction Models

Prediction is the task of estimating the value of a target attribute for a given instance based on the values of other

attributes (or input attributes) for that instance. It is the problem that supervised ML algorithms solve: they generate prediction models. The spam-filter example we used to illustrate supervised learning is also applicable here: we use supervised learning to train a spam-filter model, and the spam-filter model is a prediction model. The typical use case of a prediction model is to estimate the target attribute for new instances that are not in the training data set. Continuing our spam example, we train our spam filter (prediction model) on a data set of old emails and then use the model to predict whether new emails are spam or not spam. Prediction problems are possibly the most popular type of problem that ML is used for, so the rest of this chapter focuses on prediction as the case study for introducing ML. We begin our introduction to prediction models with a concept fundamental to prediction: *correlation analysis*. Then we explain how supervised ML algorithms work to create different types of popular prediction models, including linear-regression models, neural network models, and decision trees.

Correlations Are Not Causations, but Some Are Useful

A *correlation* describes the strength of association between two attributes.[1] In a general sense, a correlation can describe any type of association between two attributes. The term *correlation* also has a specific statistical meaning, in which it is often used as shorthand for "Pearson

correlation." A Pearson correlation measures the strength of a linear relationship between two numeric attributes. It ranges in value from −1 to +1. The letter r is used to denote the Pearson value or coefficient between two attributes. A coefficient of $r = 0$ indicates that the two attributes are not correlated. A coefficient of $r = +1$ indicates that the two attributes have a perfect positive correlation, meaning that every change in one attribute is accompanied by an equivalent change in the other attribute in the same direction. A coefficient of $r = -1$ indicates that the two attributes have a perfect negative correlation, meaning that every change in one attribute is accompanied by the opposite change in the other attribute. The general guidelines for interpreting Pearson coefficients are that a value of $r \approx \pm 0.7$ indicates a strong linear relationship between the attributes, $r \approx \pm 0.5$ indicates a moderate linear relationship, $r \approx \pm 0.3$ indicates a weak relationship, and $r \approx 0$ indicates no relationship between the attributes.

In the case of the diabetes study, from our knowledge of how humans are physically made we would expect that there will be relationships between some of the attributes listed in table 2. For example, it is generally the case that the taller someone is, the larger her shoe size is. We would also expect that the more someone exercises, the lighter she will be, with the caveat that a tall person is likely to be heavier than a shorter person who exercises the same amount. We would also expect that there will be

no obvious relationship between someone's shoe size and the amount she exercises. Figure 9 presents three scatterplots that illustrate how these intuitions are reflected in the data. The scatterplot at the top shows how the data spread out if the plotting is based on shoe size and height. There is a clear pattern in this scatterplot: the data go from the bottom-left corner to the top-right corner, indicating the relationship that as people get taller (or as we move to the right on the x axis), they also tend to wear larger shoes (we move up on the y axis). A pattern of data generally going from bottom left to top right in a scatterplot is indicative of a positive correlation between the two attributes. If we compute the Pearson correlation between shoe size and height, the correlation coefficient is $r = 0.898$, indicating a strong positive correlation between this pair of attributes. The middle scatterplot shows how the data spread out when we plot weight and exercise. Here the general pattern is in the opposite direction, from top left to bottom right, indicating a negative correlation: the more people exercise, the lighter they are. The Pearson correlation coefficient for this pair of attributes is $r = -0.710$, indicating a strong negative correlation. The final scatterplot, at the bottom, plots exercise and shoe size. The data are relatively randomly distributed in this plot, and the Pearson correlation coefficient for this pair of attributes is $r = -0.272$, indicating no real correlation.

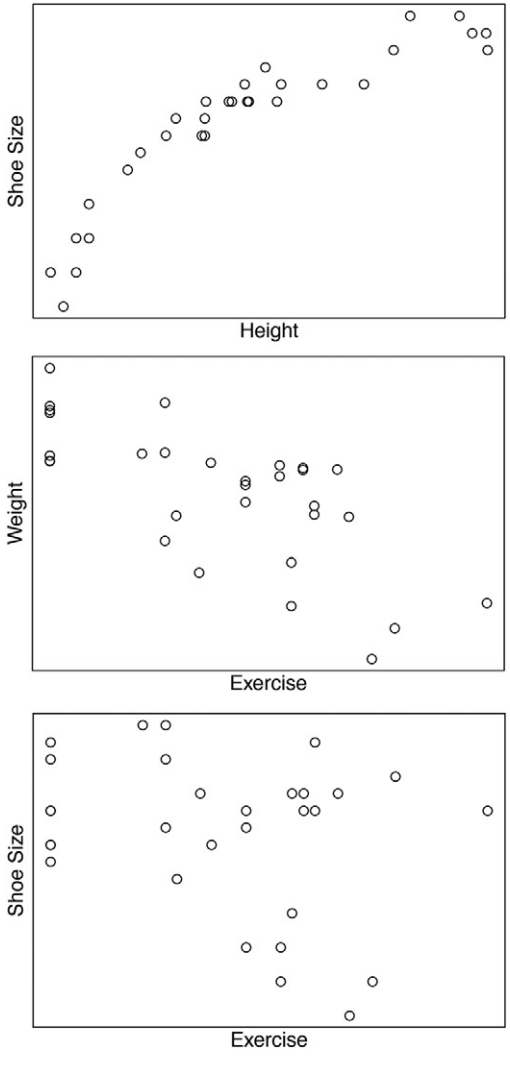

Figure 9 Scatterplots of shoe size and height, weight and exercise, and shoe size and exercise.

The fact that the definition of a statistical Pearson correlation is between two attributes might appear to limit the application of statistical correlation to data analysis to just pairs of attributes. Fortunately, however, we can circumvent this problem by using functions over sets of attributes. In chapter 2, we introduced BMI as a function of a person's weight and height. Specifically, it is the ratio of his weight (in kilograms) divided by his height (in meters) squared. BMI was invented in the nineteenth century by a Belgian mathematician, Adolphe Quetelet, and is used to categorize individuals as underweight, normal weight, overweight, or obese. The ratio of weight and height is used because BMI is designed to have a similar value for people who are in the same category (underweight, normal weight, overweight, or obese) irrespective of their height. We know that weight and height are positively correlated (generally, the taller someone is, the heavier he is), so by dividing weight by height, we control for the effect of height on weight. We divide by the square of the height because people get wider as they get taller, so squaring the height is an attempt to account for a person's total volume in the calculation. Two aspects of BMI are interesting for our discussion about correlation between multiple attributes. First, BMI is a function that takes a number of attributes as input and maps them to a new value. In effect, this mapping creates a new derived (as opposed to raw) attribute in the data. Second, because a person's BMI

is a single numeric value, we can calculate the correlation between it and other attributes.

In our case study of the causes of Type 2 diabetes in white American adult males, we are interested in identifying if any of the attributes have a strong correlation with the target attribute describing a person's likelihood of developing diabetes. Figure 10 presents three more scatterplots, each plotting the correlation between the target attribute (diabetes) and another attribute: height, weight, and BMI. In the scatterplot of height and diabetes, there doesn't appear to be any particular pattern in the data indicating that there is no real correlation between these two attributes (the Pearson coefficient is $r = -0.277$). The middle scatterplot shows the distribution of the data plotted using weight and diabetes. The spread of the data indicates a positive correlation between these two attributes: the more someone weighs, the more likely she is to develop diabetes (the Pearson coefficient is $r = 0.655$). The bottom scatterplot shows the data set plotted using BMI and diabetes. The pattern in this scatterplot is similar to the middle scatterplot: the data spread from bottom left to top right, indicating a positive correlation. In this scatterplot, however, the instances are more tightly packed together, indicating that the correlation between BMI and diabetes is stronger than the correlation between weight and diabetes. In fact, the Pearson coefficient for diabetes and BMI for this data set is $r = 0.877$.

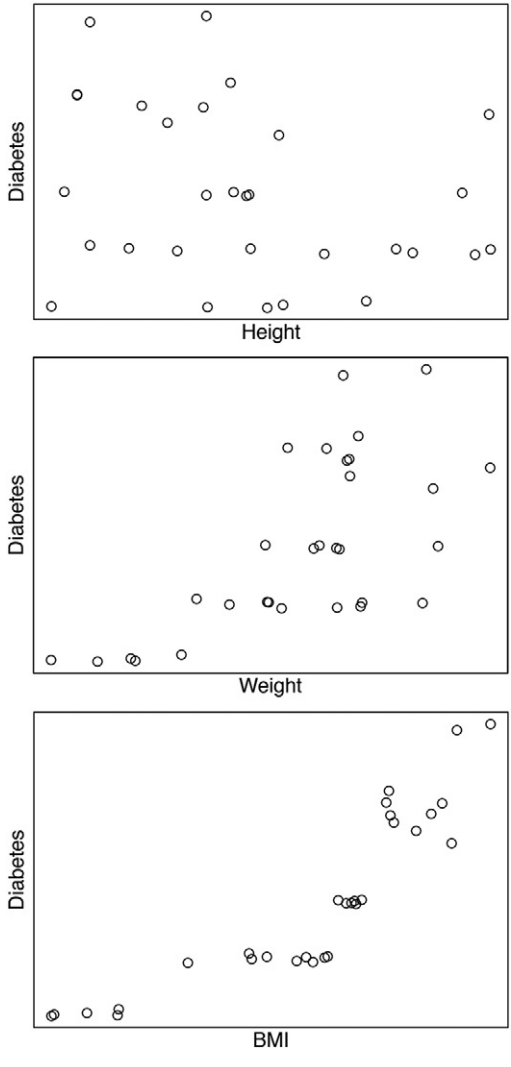

Figure 10 Scatterplots of the likelihood of diabetes with respect to height, weight, and BMI.

The BMI example illustrates that it is possible to create a new derived attribute by defining a function that takes multiple attributes as input. It also shows that it is possible to calculate a Pearson correlation between this derived attribute and another attribute in the data set. Furthermore, a derived attribute can actually have a higher correlation with a target attribute than any of the attributes used to generate the derived attribute have with the target. One way of understanding why BMI has a higher correlation with the diabetes attribute compared to the correlation for either height or weight is that the likelihood of someone developing diabetes is dependent on the interaction between height and weight, and the BMI attribute models this interaction appropriately for diabetes. Clinicians are interested in people's BMI because it gives them more information about the likelihood of someone developing Type 2 diabetes than either just the person's height or just his weight does independently.

We have already noted that attribute selection is a key task in data science. So is attribute design. Designing a derived attribute that has a strong correlation with an attribute we are interested in is often where the real value of data science is found. Once you know the correct attributes to use to represent the data, you are able to build accurate models relatively quickly. Uncovering and designing the right attributes is the difficult part. In the case of BMI, a human designed this derived attribute in the nineteenth

century. However, ML algorithms can learn interactions between attributes and create useful derived attributes by searching through different combinations of attributes and checking the correlation between these combinations and the target attribute. This is why ML is useful in contexts where many weak interacting attributes contribute to the process we are trying to understand.

Identifying an attribute (raw or derived) that has a high correlation with a target attribute is useful because the correlated attribute may give us insight into the process that causes the phenomenon the target attribute represents: the fact that BMI is strongly correlated with the likelihood of a person's developing diabetes indicates that it is not weight by itself that contributes to a person's developing diabetes but whether that person is overweight. Also, if an input attribute is highly correlated with a target attribute, it is likely to be a useful input into the prediction model. Similar to correlation analysis, prediction involves analyzing the relationships between attributes. In order to be able to map from the values of a set of input attributes to a target attribute, there must be a correlation between the input attributes (or some derived function over them) and the target attribute. If this correlation does not exist (or cannot be found by the algorithm), then the input attributes are irrelevant for the prediction problem, and the best a model can do is to ignore those inputs and always predict the central tendency of that target[2] in the data set.

Conversely, if a strong correlation does exist between input attributes and the target, then it is likely that an ML algorithm will be able to generate a very accurate prediction model.

Linear Regression

When a data set is composed of numeric attributes, then prediction models based on regression are frequently used. *Regression analysis* estimates the expected (or average) value of a numeric target attribute when all the input attributes are fixed. The first step in a regression analysis is to hypothesize the structure of the relationship between the input attributes and the target. Then a parameterized mathematical model of the hypothesized relationship is defined. This parameterized model is called a *regression function*. You can think of a regression function as a machine that converts inputs to an output value and of the parameters as the settings that control the behavior of a machine. A regression function may have multiple parameters, and the focus of regression analysis is to find the correct settings for these parameters.

It is possible to hypothesize and model many different types of relationships between attributes using regression analysis. In principle, the only constraint on the structure of the relationship that can be modeled is the ability to define the appropriate regression function. In some domains, there may be strong theoretical reasons to assert a

particular type of relationship, but in the absence of this type of domain theory it is good practice to begin by assuming the simplest form of relationship—namely, a linear relationship—and then, if need be, progress to model more complex relationships. One reason for starting with a linear relationship is that linear-regression functions are relatively easy to interpret. The other reason is the commonsense notion that keeping things as simple as possible is generally a good idea.

When a linear relationship is assumed, the regression analysis is called *linear regression*. The simplest application of linear regression is modeling the relationship between two attributes: an input attribute X and a target attribute Y. In this simple linear-regression problem, the regression function has the following form:

$$Y = \omega_0 + \omega_1 X$$

This regression function is just the equation of a line (often written as $y = mx + c$) that is familiar to most people from high school geometry.[3] The variables ω_0 and ω_1 are the parameters of the regression function. Modifying these parameters changes how the function maps from the input X to the output Y. The parameter ω_0 is the y-intercept (or c in high school geometry) that specifies where the line crosses the vertical y axis when X is equal to zero. The parameter ω_1 defines the slope of the line (i.e., it is equivalent to m in the high school version).

In regression analysis, the parameters of a regression function are initially unknown. Setting the parameters of a regression function is equivalent to searching for the line that best fits the data. The strategy for setting these parameters begins by guessing parameters values and then iteratively updating the parameters so as to reduce the overall error of the function on the data set. The overall error is calculated in three steps:

1. The function is applied to the data set, and for each instance in the data set it estimates the value of the target attribute.

2. The error of the function for each instance is calculated by subtracting the estimated value of the target attribute from the actual value of the target attribute.

3. The error of the function for each instance is squared, and then these squared values are summed.

The error of the function for each instance is squared in step 3 so that the error in the instances where the function overestimates the target doesn't cancel out with the error when it underestimates. Squaring the error makes the error positive in both cases. This measure of error is known as the *sum of squared errors* (SSE), and the strategy of fitting a linear function by searching for the parameters that minimize the SSE is known as *least squares*. The SSE is defined as

$$SSE = \sum_{i=i}^{n} (target_i - prediction_i)^2$$

where the data set contains n instances, $target_i$ is the value of the target attribute for instance i in the data set, and $prediction_i$ is the estimate of the target by function for the same instance.

To create a linear-regression prediction model that estimates the likelihood of an individual's developing diabetes with respect to his BMI, we replace X with the BMI attribute, Y with the diabetes attribute, and apply the least-squares algorithm to find the best-fit line for the diabetes data set. Figure 11a illustrates this best-fit line and where it lies relative to the instances in the data set. In figure 11b, the dashed lines show the error (or residual) for each instance for this line. Using the least-squares approach, the best-fit line is the line that minimizes the sum of the squared residuals. The equation for this line is

$$Diabetes = -7.38431 + 0.55593 * BMI\,.$$

The slope parameter value $\omega_1 = 0.55593$ indicates that for each increase of one unit in BMI, the model increases the estimated likelihood of a person developing diabetes by a little more than half a percent. In order to predict the likelihood of a person's developing diabetes, we simply input his BMI into the model. For example, when BMI equals 20, the model returns a prediction of a 3.73 percent